Finalis: A Finalized-Tip Byzantine Settlement Protocol

Technical Whitepaper
October 2026 | reikagitakashi@gmail.com

Abstract

Finalis is a blockchain protocol designed around deterministic finality rather than fork-choice competition. The live protocol advances only at `finalized_height + 1`. A transition is committed when a quorum certificate over `(height, round, transition_id)` is verified against the canonical committee for that context. The protocol removes non-finalized chain selection, derives committee checkpoints deterministically from finalized state, and replays state from authoritative finalized artifacts.

The design objective is strict settlement semantics with bounded validation cost, explicit replay authority, and auditable consensus behavior under standard Byzantine assumptions.

1. Introduction

Classical proof-of-work chains expose probabilistic finality through depth and fork choice. Many BFT systems add explicit finality but still carry complex runtime branches around pending forks and speculative views.

Finalis adopts a narrower model:

- There is one authoritative finalized tip.
- Consensus work is meaningful only for `height = finalized_height + 1`.
- Deterministic replay from finalized artifacts reconstructs canonical state.

This model is intended for systems that prioritize unambiguous settlement over speculative throughput.

2. Model and Notation

Let:

- `H_f` be current finalized height.
- `T_h` be the frontier transition proposed for height `h`.
- `id(T_h)` be transition identifier.
- `C(h, r)` be canonical committee for height `h`, round `r`.
- `N = |C(h, r)|` and `Q = floor(2N/3) + 1`.

A vote signs `sha256d("SC-VOTE-V1" || D || h || r || id(T_h))`. A timeout vote signs `sha256d("SC-TIMEOUT-V1" || D || h || r)`, where `D = sha256d("SC-CHAIN-DOMAIN-V1" || network_id ||`

`genesis_hash)` binds the signature to this chain; `Q` timeout votes for `(h, r)` form a timeout certificate `TC(h, r)`. Validator signatures are Ed25519.

The committee is taken from the finalized checkpoint for the epoch containing `h` (mainnet epochs are 32 blocks). It is the same for every round of a height, except in the degenerate single-member case, where a fallback member is selected per round.

A finality certificate contains:

- `height`, `round`, `transition_id`
- committee members used for verification
- canonicalized valid signatures (deduplicated, sorted, quorum-truncated)

3. Consensus Rule

A transition `T_h` is finalized iff:

1. `h = H_f + 1`
2. certificate payload matches `T_h`
3. committee context is valid for `(h, r)`
4. at least `Q` distinct committee precommit signatures verify on the precommit message

The runtime rejects work outside `H_f + 1` on the live path.

Finality is two-phase, following Tendermint [Buchman, Kwon, Milosevic 2018], with timeout certificates (TC) in place of nil votes. The leader of `(h, r)` signs a proposal; validators **prevote** its `transition_id`; a quorum of prevotes for one `(h, r, id)` (a **polka**) makes a validator lock on `id` at `r` and **precommit** it; `Q` precommits for one `(h, r, id)` finalize it. Prevotes and precommits use separate signing domains.

Rounds advance on a TC. A proposal for round `r > 0` carries a TC or a polka from a lower round. A leader that has seen a polka (its *valid value*) re-proposes that transition unchanged with the polka as proof of lock (`pol`); otherwise it builds a fresh transition at `r`.

A validator prevotes a proposal `P` in round `r` at most once, and only if it is unlocked, locked on `id(P)`, or `P` carries a polka from a round `≥` its lock round. It precommits only on a polka in its current round, after its lock is durable. Lock, valid value and signed prevotes are persisted before the corresponding message is sent; a validator whose safety state is unreadable abstains at that height. A validator never votes in a round it has already sent a timeout vote for.

A validator prevotes and precommits only if it holds, in its local store, a valid ingress certificate for every record of the proposed slice; otherwise it defers and fetches the missing records from peers. A finality certificate therefore implies that at least `f + 1` honest validators hold the finalized slice's certified records.

One transition can be finalized by precommits from different rounds on different nodes, so the child transition names the parent's commit round (`prev_finality_round`) alongside its participation record `prev_finality_signers`.

3.1 Safety intuition

Two conflicting finalized transitions at the same `(h, r)` require two quorum signer sets over different payloads. Quorum intersection implies at least one overlapping honest signer would need to sign conflicting messages, violating Byzantine assumptions. Across rounds, the honest precommitters of a finalized transition are locked on it, and a lock moves only to a later polka, which those locked validators never help form for anything else (Proposition 3).

3.2 Liveness assumptions

Liveness remains conditional on:

- eventual synchrony
- quorum participation
- valid proposal propagation

Failure of these conditions causes halt/degradation, not protocol-level ambiguous finalization.

3.3 Numbered propositions

Proposition 1 (Single-Context Safety).
Under quorum intersection and honest non-equivocation assumptions, two distinct transitions cannot both finalize at the same `(height, round)` context.

Proof sketch. Distinct finalized transitions at one `(h, r)` imply two quorum signature sets over different payloads. Quorum intersection yields at least one common honest signer, contradicting non-equivocation.

Proposition 2 (Context Binding).
A valid vote for `(h, r, id(T_h))` cannot be reused as a valid vote for a different `(h', r', id(T_{h'}))`.

Proof sketch. Signature verification binds the exact message tuple `(height, round, transition_id)`. Any tuple change changes the verified message.

Proposition 3 (Cross-Round Safety).
With `n = 3f + 1` committee members, at most `f` Byzantine, no two distinct transitions finalize at the same height, in any rounds.

Proof sketch. Let `T` finalize with precommits at round `r`, and let `S` be the honest precommitters; `|S| ≥ f + 1`, and each locked `T` at `r` after seeing a polka for `T` at `r`. A finality certificate for `U ≠ T` at round `r' ≥ r` needs a polka for `U` at `r'`. At `r` that would need an honest validator to prevote twice in one round (quorum intersection). For `r' > r`, let `r*` be the first round above `r` with a polka for some `U ≠ T`. Its `2f + 1` prevotes cannot all come from the `≤ 2f` validators outside `S`, so some member of `S`, locked on `T` at a round `ℓ ≥ r`, prevoted `U` at `r*`. It does so only for a proposal carrying a polka for `U` at a round in `[ℓ, r*)`, and no such polka exists (none at `r`, none in `(r, r*)` by minimality of `r*`). Contradiction. A finality certificate for `U` at `r' < r` is covered by the symmetric argument with the roles of `T` and `U` exchanged. Full argument: `docs/spec/TWO_PHASE_FINALITY.md` §4.

4. Certified Ingress Layer

Finalis executes ordered ingress records into transitions. Each certified ingress record binds a transaction payload to lane and sequence context.

An ingress certificate needs only one valid committee signature; it fixes a record's lane position and attributes it to a committee member. It is not a finality artifact: a record becomes canonical only through a finalized transition, and honest validators vote for a transition only after verifying the certificates of all its records (§3).

Ingress validity requires:

- certificate epoch equals `committee_epoch_start(finalized_height + 1)`
- at least one signature; every signature valid; no duplicate signers
- signer committee membership when committee context is available
- payload parse success with exact `txid` and `tx_hash` match
- lane assignment recomputes and matches certificate lane
- strict sequence continuity and `prev_lane_root` chaining

One committee member certifies each lane per epoch, chosen by `sha256d(epoch_start || lane)` modulo the committee size, so a single writer extends each lane. The choice is routing only (a certificate from any committee member is valid), and it changes every epoch, so a certifier that censors its lanes holds them for at most one epoch. Validators re-forward uncertified mempool transactions to the current certifier.

Stale-epoch ingress is rejected. Equivocation at fixed `(epoch, lane, seq)` is rejected and persisted as deterministic evidence. Re-delivery of a record whose certificate is identical to the one already stored (for example, a record first received by gossip and then again in a range-sync response) is an idempotent no-op, not a sequence violation; genuine gaps remain rejected.

Proposition 4 (Ingress Epoch Freshness).
Certified ingress from a stale epoch cannot enter canonical execution.

Proof sketch. Ingress validation enforces `certificate.epoch = committee_epoch_start(finalized_height + 1)`. Mismatch is rejected pre-execution.

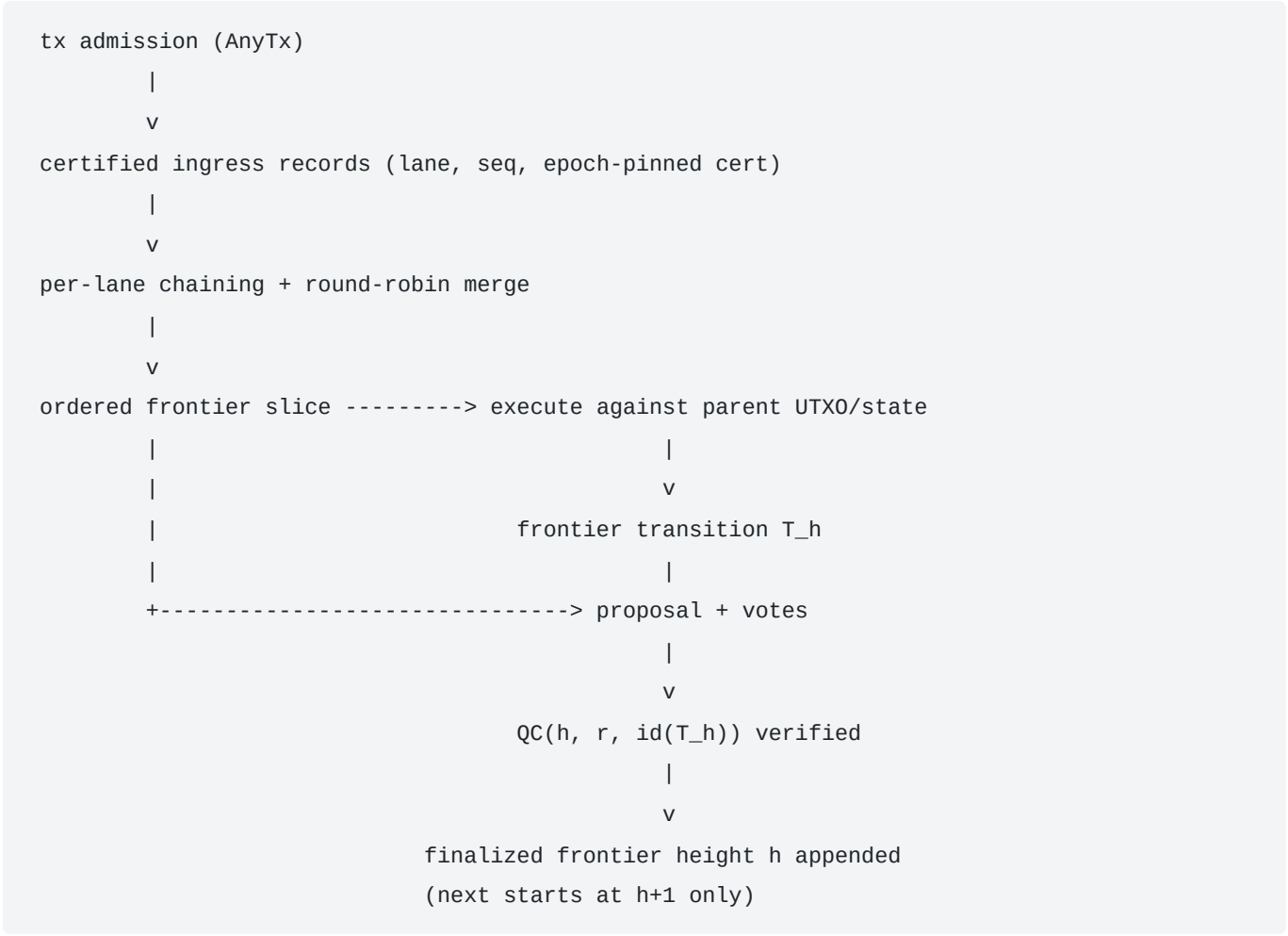
5. Deterministic State Transition

Canonical replay is defined as:

`S_h = ApplyFinalizedRecord(S_{h-1}, R_h)`

where `R_h` is the finalized record at height `h`.

5.1 Block and Frontier Append Path



Authoritative replay inputs are:

- genesis/network identity
- finalized frontier transitions in height order
- canonical finality certificate per finalized height
- finalized ingress artifacts required for transition verification

Non-authoritative caches may be rebuilt and cannot change canonical output.

Proposition 5 (Replay Uniqueness).
For fixed authoritative finalized inputs, canonical derived state is unique.

Proof sketch. Replay applies a deterministic transition function in height order over canonical finalized records. No alternate fork-choice branch is in the live model.

6. Committee Checkpoints

Committee source is finalized-state-derived checkpoint metadata at epoch boundaries. Live proposal/vote handling consumes this output rather than recomputing policy heuristics locally.

Adaptive checkpoint parameters are deterministic from finalized qualified operator depth:

- target committee size
- minimum eligible operators
- checkpoint minimum bond

Fallback mode is explicit when eligibility falls below threshold; fallback metadata is part of deterministic checkpoint state.

7. Transaction and Script Semantics

Finalis supports version-aware execution:

- `Tx` (legacy transparent)
- `TxV2` (confidential-capable)
- runtime dispatch via `AnyTx`

Validator-control scripts (`SCONBREG`, `SCVALJRQ`, `SCVALREG`) are enforced with aligned semantics across legacy `Tx` and transparent outputs in `TxV2`.

Validation hardening includes:

- explicit input/output sum overflow checks
- max-fee policy enforcement on both V1 and V2 paths
- V2 fee validation: transparent inputs must cover transparent outputs + fee; confidential value conservation is enforced via commitment balance checks

Proposition 6 (Script-Parity Invariant).
Validator-control script semantics are consistent across legacy `Tx` and transparent outputs in `TxV2`.

Proof sketch. Validation dispatch uses shared script-semantic checks for `SCONBREG`, `SCVALJRQ`, and `SCVALREG` invariants.

7.1 Confidential Outputs

Confidential UTXOs are enabled from genesis. A confidential output carries a Pedersen value commitment, a one-time public key derived per receive request, and a range proof. A `TxV2` balances when its commitments, transparent values, and fee sum to an excess commitment, and that excess is authorized by a BIP340 Schnorr signature under the excess blind. Each confidential input is authorized by a Schnorr signature under its one-time spend key. Only amounts are hidden, and only where both sides of a transfer are confidential: the transaction graph, fees and the amounts of shield and unshield transactions remain public.

Consensus bounds on confidential work:

- at most 16 confidential inputs and 12 confidential outputs per transaction; 12 canonical range proofs (at most 5,134 bytes each) fit the 65,536-byte per-transaction proof budget, and 13 do not
- a structural verify weight per `TxV2`, computed before any cryptographic verification: 64 per confidential input, range-proof bytes + 256 per confidential output, 64 for a non-identity excess, plus a 256 per-

transaction base when any of these apply; the unit is one range-proof byte (about 0.72 µs of verification, calibrated by benchmark)
- a block-level cap of 2,000,000 verify weight over every parseable `TxV2` in the ordered frontier slice, including transactions later rejected; a slice above the cap is an invalid transition

Independently of confidential work, a frontier slice is an invalid transition above 1,000 records or 1 MiB of raw transaction bytes.

Proposition 7 (Confidential Turnstile).
The value held in confidential outputs never goes negative in canonical state.

Proof sketch. Canonical derived state commits a pool value `P`, updated only from public data: each accepted `TxV2` adds its transparent inputs minus transparent outputs minus fee. Frontier execution rejects any `TxV2` that would make `P` negative, so a forged range proof cannot withdraw more transparent value than has entered the confidential pool.

The reference wallet draws fresh auxiliary randomness from the operating-system CSPRNG for every confidential Schnorr signature (spend and excess). This is wallet hygiene, not a consensus rule: BIP340 signatures stay valid for any auxiliary input.

8. Economics and Incentives

Economics is separated into two deterministic planes:

1. Height-gated economics policy (`active_economics_policy(network, height)`) for reward/ticket parameters.
2. Adaptive checkpoint control plane for committee target/eligibility/bond thresholds.

Emission is finite and deterministic in current implementation:

- total primary emission: `7,000,000 FLS`
- emission horizon: `2,102,400` blocks (12 years at a 180-second block target)
- annual issuance declines by 20% year over year; yearly budgets sum exactly to the cap
- reserve accrual during emission: `30%` of gross issuance; of the remaining 70%, `3%` is carved into onboarding rewards when eligible onboarding recipients exist

From block 1, every finalized fee is pooled into its epoch: half is paid to the epoch's validators at settlement and half returns to the reserve, as does the value of slashed bonds. After the cap, new issuance is zero, and each epoch pays out a fixed fraction of the reserve's balance, 5% a year, so the reserve never empties.

9. Ticket PoW Boundary

Ticket PoW is secondary and bounded:

- one bounded search per operator over nonces `[0, 4095]`
- difficulty clamped to 8–12 bits

- bounded bonus capped by `ticket_bonus_cap_bps` of the active economics policy

Ticket PoW does not define finality and does not bypass admission controls. Admission PoW for onboarding/join scripts is a separate mechanism validated in script semantics.

10. Security Boundaries

Finalis security requires:

- signature unforgeability
- deterministic serialization and hashing
- quorum intersection assumptions
- deterministic checkpoint and replay rules

The protocol intentionally prefers safe halt over speculative reconstruction when authoritative finalized artifacts are missing or inconsistent.

Proposition 8 (Fail-Closed Recovery).
Missing/inconsistent authoritative finalized artifacts lead to halt, not speculative canonical continuation.

Proof sketch. Startup/replay requires canonical finalized artifacts and consistency checks; violations are terminal in consensus-critical paths.

11. Operational Interpretation

For integrators and exchanges:

- settlement decisions must use finalized state only
- relay/mempool acceptance is not a settlement signal
- finalized inclusion is the credit-safe condition

No confirmation-depth heuristic is required by protocol semantics.

12. Conclusion

Finalis frames blockchain consensus as deterministic finalized-state progression rather than fork-choice competition. By constraining live processing to `finalized_height + 1`, binding finality to quorum certificates over exact transition context, and deriving committee and replay state from finalized artifacts, the protocol aims to provide auditable, bounded, and unambiguous settlement behavior.

Appendix A. Threat Model (Short Form)

Adversary classes considered:

- Byzantine validators: equivocation, withholding, malformed votes/proposals.

- Network adversary: delay, drop, partition, eclipse.
- Storage adversary: cache corruption, partial persistence, artifact deletion.
- Economic/Sybil adversary: operator fragmentation, capital concentration, bounded ticket optimization.

Out-of-scope guarantees:

- real-world identity uniqueness of `operator_id`
- off-chain governance/social coordination guarantees
- wallet/explorer UX correctness as a safety primitive

Security posture:

- safety prioritized over availability during inconsistency
- deterministic replay prioritized over speculative reconstruction
- bounded validation and explicit artifact authority prioritized over heuristic acceptance

References

- E. Buchman, J. Kwon, Z. Milosevic, "The latest gossip on BFT consensus", 2018 (arXiv:1807.04938)
- `docs/spec/TWO_PHASE_FINALITY.md`
- `docs/PROTOCOL-SPEC.md`
- `docs/CONSENSUS.md`
- `docs/LIVE_PROTOCOL.md`
- `docs/ECONOMICS.md`
- `docs/POW-AND-DIFFICULTY.md`
- `docs/REWARD-SETTLEMENT.md`
- `docs/ONBOARDING-PROTOCOL.md`
- `docs/ADVERSARIAL_MODEL.md`
- `docs/spec/CHECKPOINT_DERIVATION_SPEC.md`
- `docs/spec/AVAILABILITY_STATE_COMPLETENESS.md`
- `docs/spec/CONFIDENTIAL_UTXO_SPEC.md`